STUDENT EXAM PERFORMANCE TRACKER

1. Problem Statement

Students write multiple examinations during an academic year. Comparing examination marks manually can be time-consuming and difficult. It is important to understand whether a student's academic performance is improving, declining, or remaining unchanged.

The Student Exam Performance Tracker is a Python program designed to collect a student's name and marks in three examinations. It calculates the average score, determines the difference between the first and last examination scores, and identifies the student's overall performance trend.

2. Objectives

1. To develop a simple student performance tracking system using Python.
2. To accept the student's name as input.
3. To collect marks from three examinations.
4. To calculate the average marks obtained by the student.
5. To calculate the change in marks between the first and last examinations.
6. To identify whether the student's performance has improved, declined, or remained unchanged.
7. To display the student's performance details clearly.

3. Assumptions

1. The program accepts one student's information at a time.
2. The student must enter their name.
3. Exactly three examination scores are entered.
4. Examination scores must be numeric values.
5. The program assumes that the entered marks are valid.
6. The program compares only the first and third examination scores to determine the performance trend.
7. The program does not store student information permanently.
8. The program does not perform validation for maximum or minimum marks.

4. Algorithm

Step 1: Start the program.

Step 2: Read the student's name.

Step 3: Create an empty list to store examination scores.

Step 4: Repeat the following steps three times:

- Accept an examination score.
- Convert the score into a floating-point number.
- Add the score to the list.

Step 5: Calculate the average of all three examination scores.

Step 6: Calculate the change in performance by subtracting the first examination score from the third examination score.

Step 7: Check the performance trend:

- If the change is greater than zero, display "Improved".
- If the change is less than zero, display "Declined".
- Otherwise, display "Unchanged".

Step 8: Display the student's name, examination scores, average score, score change, and performance trend.

Step 9: Stop the program.

5. Flowchart

```mermaid
flowchart TD
```

```
    A([Start]) --> B[/Enter student name/]
    B --> C[Create empty scores list]
    C --> D[/Enter exam score/]
    D --> E[Append score to list]
    E --> F{Three scores entered?}
    F -- No --> D
    F -- Yes --> G[Calculate average]
    G --> H[Calculate last score - first score]
    H --> I{Is change greater than 0?}
    I -- Yes --> J[Trend = Improved]
    I -- No --> K{Is change less than 0?}
    K -- Yes --> L[Trend = Declined]
    K -- No --> M[Trend = Unchanged]
    J --> N[Display student details]
    L --> N
    M --> N
    N --> O([Stop])
```

6. Pseudocode

```text
BEGIN

    INPUT student_name

    CREATE empty list scores

    FOR exam = 1 TO 3 DO
        INPUT score
        APPEND score TO scores
    END FOR
```

```
    average = SUM(scores) / LENGTH(scores)

    change = LAST(scores) - FIRST(scores)

    IF change > 0 THEN
        trend = "Improved"

    ELSE IF change < 0 THEN
        trend = "Declined"

    ELSE
        trend = "Unchanged"

    END IF

    DISPLAY student_name
    DISPLAY scores
    DISPLAY average
    DISPLAY change
    DISPLAY trend

END
```

7. Python Source Code

```python
name = input("Student name: ")
scores = []

for exam in range(3):
```

```
    score = float(input("Enter exam " + str(exam + 1) + " score: "))
    scores.append(score)

average = sum(scores) / len(scores)
change = scores[-1] - scores[0]

if change > 0:
    trend = "Improved"
elif change < 0:
    trend = "Declined"
else:
    trend = "Unchanged"

print("\\nStudent:", name)
print("Scores:", scores)
print("Average score:", round(average, 2))
print("Change from first to last exam:", round(change, 2))
print("Performance trend:", trend)
```

8. Test Cases

The following test cases verify the program under different examination performance conditions.

Test case	Exam 1	Exam 2	Exam 3	Expected trend
1	60	70	80	Improved
2	90	75	60	Declined
3	70	80	70	Unchanged
4	45	65	85	Improved
5	80	60	50	Declined

Test Case 1: Improved Performance

Input:

```text
Student name: Bhupesh
Enter exam 1 score: 60
Enter exam 2 score: 70
Enter exam 3 score: 80
```

Output:

```text
\nStudent: Bhupesh
Scores: [60.0, 70.0, 80.0]
Average score: 70.0
Change from first to last exam: 20.0
Performance trend: Improved
```

Test Case 2: Declined Performance

Input:

```text
Student name: Rahul
Enter exam 1 score: 90
Enter exam 2 score: 75
Enter exam 3 score: 60
```

Output:

```text
\nStudent: Rahul
```

Scores: [90.0, 75.0, 60.0]

Average score: 75.0

Change from first to last exam: -30.0

Performance trend: Declined

```

### Test Case 3: Unchanged Performance

**Input:**

```text

Student name: Priya

Enter exam 1 score: 70

Enter exam 2 score: 80

Enter exam 3 score: 70

```

Output:

```text

\nStudent: Priya

Scores: [70.0, 80.0, 70.0]

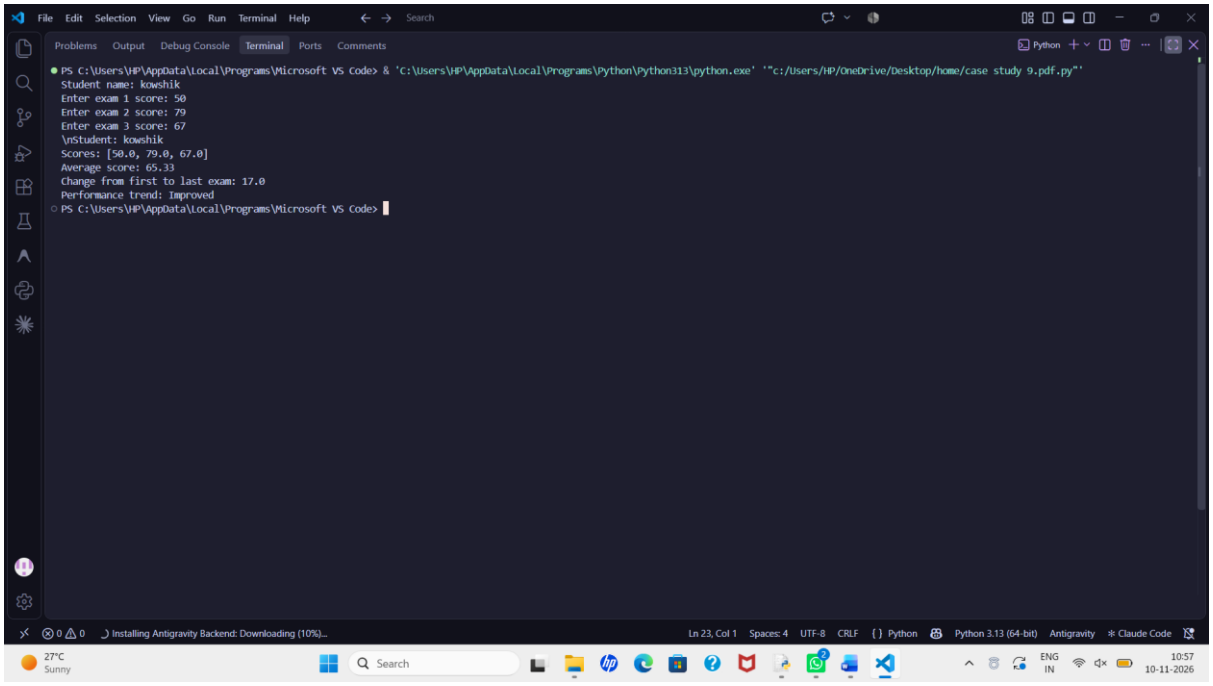
Average score: 73.33

Change from first to last exam: 0.0

Performance trend: Unchanged

```

Output Screenshot Instructions
```



9. Conclusion

The Student Exam Performance Tracker was successfully developed using Python. The program accepts student details and examination scores, calculates the average marks, determines the difference between the first and last examination scores, and identifies the student's performance trend.

This project demonstrates the practical use of Python concepts such as input and output operations, lists, loops, conditional statements, arithmetic operations, and built-in functions.

The project helps students understand their academic performance and provides a simple way to compare examination results. It can be further enhanced by adding multiple student records, input validation, data storage, graphical reports, and a user-friendly interface.

Thus, the project successfully achieves its objective of tracking and analyzing student examination performance using Python.